$ Aide-mémoire Kubernetes

Les commandes et notions essentielles de Kubernetes, groupées par usage.
Chaque partie d'une commande garde toujours la même couleur pour qu'on la repère d'un coup d'œil.

```
commande racine sous-commande option / flag valeur à remplacer
```

Qu'est-ce que Kubernetes (K8s) ?

Kubernetes est un système open source qui **automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées**. Là où Docker lance des conteneurs sur une machine, Kubernetes les orchestre sur un ensemble de machines (un *cluster*) : il décide où les placer, en relance automatiquement en cas de panne et en ajoute ou en retire selon la charge.

L'abréviation **K8s** vient du nombre de lettres entre le « K » et le « s » de Kubernetes (8). On pilote un cluster avec l'outil en ligne de commande `kubectl`.

En pratique, Kubernetes aide à **lancer et gérer automatiquement de nombreux conteneurs**. Exemple : une application découpée en microservices (`users`, `products`, `orders`, `notifications`), chacun dans son conteneur Docker, à faire tourner sur plusieurs serveurs. Plutôt que de choisir à la main sur quel serveur lancer chaque conteneur, on décrit à Kubernetes ce qu'on veut obtenir, et il répartit lui-même les conteneurs sur les serveurs disponibles.

Ce que Kubernetes apporte

• **Déploiement automatique** (*Automatic Deployment*) : il simplifie le déploiement sur de nombreuses machines.
• **Disponibilité** (*Availability*) : il s'assure que les applications tournent toujours, même en cas de panne. Si un conteneur plante (ex. `users`), Kubernetes le détecte et en relance automatiquement un nouveau.
• **Mise à l'échelle** (*Scalability*) : il ajoute ou retire des instances d'un service selon le trafic. Si la charge augmente sur `products`, il lance des copies supplémentaires de ce conteneur, puis les retire quand la charge baisse.
• **Répartition de charge** (*Load balancing*) : il répartit le trafic entrant entre les différentes instances d'un même service, pour qu'aucune ne soit surchargée.

Un peu d'histoire

Kubernetes a été développé par **Google**, qui l'a rendu **open source en 2014**, puis confié à la **Cloud Native Computing Foundation (CNCF)**, qui assure aujourd'hui sa gouvernance.

Orchestration de conteneurs

L'**orchestration de conteneurs** est le processus qui consiste à **gérer automatiquement de nombreux conteneurs** : les **démarrer**, les **arrêter**, les **redémarrer** et les **mettre à l'échelle** (en ajouter ou en retirer). Kubernetes est l'outil d'orchestration le plus répandu.

Les fonctionnalités d'orchestration de Kubernetes

• **Déploiement automatisé** (*Automated deployment*) : on décrit l'état voulu (quelle image, combien d'instances), Kubernetes s'occupe de le mettre en place.
• **Mise à l'échelle** (*Scaling*) : augmenter ou réduire le nombre d'instances d'un service, manuellement ou selon la charge.
• **Planification** (*Scheduling*) : choisir sur quel serveur placer chaque conteneur, selon les ressources disponibles. Exemple : plusieurs instances de `users` réparties sur différents serveurs, `products` regroupé sur un autre.
• **Auto-réparation** (*Self-healing*) : relancer automatiquement un conteneur qui plante, ou le déplacer si un

serveur tombe en panne.
• **Répartition de charge** (*Load balancing*) : répartir le trafic entre les instances d'un même service.
• **Mises à jour progressives** (*Rolling updates*) : passer à une nouvelle version de l'application en remplaçant les instances une par une, sans interruption de service.
• **Retours arrière** (*Rollbacks*) : revenir rapidement à la version précédente si la nouvelle pose problème.
• **Découverte de services** (*Service Discovery*) : permettre aux services de se trouver par leur nom, sans connaître les adresses IP des conteneurs, qui changent à chaque redémarrage. C'est essentiel dans une **architecture microservices**, où les services s'appellent en permanence : `orders` appelle simplement `http://users/…`, et Kubernetes l'oriente vers une instance disponible de `users`, quel que soit le serveur où elle tourne. Kubernetes intègre ce mécanisme nativement (via ses *Services* et son DNS interne), là où une application Spring Cloud classique utilise un annuaire dédié comme Eureka.

Architecture d'un cluster

Pod

Le **Pod** est la **plus petite unité** que Kubernetes déploie et gère. Il enveloppe un conteneur (parfois plusieurs, très liés), par exemple celui d'une application Spring Boot, et lui fournit :
• un **stockage partagé** (volumes accessibles à tous les conteneurs du Pod) ;
• un **réseau partagé** : une adresse IP propre au Pod, ses conteneurs communiquent entre eux via `localhost` ;
• des **métadonnées** sur l'environnement dans lequel il tourne (nom, labels, namespace…).
On ne lance donc pas directement des conteneurs dans Kubernetes, mais des Pods.
En pratique, un Pod contient **le plus souvent un seul conteneur** ; plusieurs conteneurs ne se regroupent dans un même Pod que s'ils doivent vraiment fonctionner ensemble (ex. une application et un conteneur annexe qui collecte ses logs). **Chaque Pod reçoit sa propre adresse IP** dans le cluster (ex. `10.1.0.160`), ce qui permet aux Pods de communiquer directement entre eux, même sur un même Node.
Les conteneurs d'un même Pod peuvent **partager des volumes de stockage** : un volume monté dans le Pod est accessible par chacun de ses conteneurs, et peut s'appuyer sur un stockage externe (ex. stockage cloud) pour que les données survivent au Pod.
Enfin, les Pods sont **éphémères** : ils peuvent être supprimés et recréés à tout moment (panne, mise à jour, mise à l'échelle), et un Pod recréé reçoit une **nouvelle adresse IP**. On ne s'appuie donc jamais sur l'IP d'un Pod ni sur les données écrites dans son conteneur : on passe par un nom de service stable, et par des volumes pour les données à conserver.

Node (nœud de travail)

Un **Node** est une machine (physique ou virtuelle) du cluster qui **exécute les Pods**. Un cluster en compte généralement plusieurs : si un Node tombe en panne, Kubernetes recrée ses Pods sur un autre Node encore disponible (auto-réparation). Chaque Node fait tourner trois composants :
• **`kubelet`** : l'agent du Node. Il démarre et arrête les conteneurs, vérifie qu'ils restent en bonne santé, leur attache le stockage dont ils ont besoin, et rend compte de ce qui tourne au Control Plane ;
• **`kube-proxy`** : gère les règles réseau du Node pour que le trafic destiné à un service atteigne le bon Pod (base de la découverte de services et de la répartition de charge) ;
• le **runtime** de conteneurs (ex. `containerd`) : le logiciel qui exécute réellement les conteneurs à partir des images.

Control Plane (plan de contrôle)

Le **Control Plane** est un ensemble de **composants de gestion** qui tournent sur une ou plusieurs machines dédiées (souvent trois, pour qu'il reste disponible si l'une tombe). C'est le « cerveau » du cluster : il reçoit les demandes (par exemple via `kubectl`), décide sur quel Node placer chaque Pod, et surveille en permanence que l'état réel correspond à l'état voulu. Il communique en continu avec le `kubelet` de chaque Node.

Ses composants :
• **`kube-api-server`** : expose l'**API Kubernetes**. C'est le point d'entrée unique du cluster : `kubectl`, les autres composants du Control Plane et le `kubelet` de chaque Node passent tous par lui ;
• **`etcd`** : base de données **clé-valeur hautement disponible** qui stocke toutes les données du cluster (état voulu, état réel, configuration). Seul le `kube-api-server` y lit et y écrit ;
• **`kube-scheduler`** : surveille les **Pods nouvellement créés** qui n'ont pas encore de Node, et choisit le Node où les placer. Il tient compte des besoins en ressources (CPU, mémoire), des règles d'affinité et d'anti-affinité (placer des Pods ensemble ou au contraire les séparer), de la localité des données, des interférences entre charges de

travail et des délais ;
• `kube-controller-manager` : exécute les **contrôleurs**, des boucles qui comparent en permanence l'état réel à l'état voulu et corrigent les écarts (ex. recréer un Pod disparu pour garder le bon nombre d'instances) ;
• `cloud-controller-manager` : fait le lien avec l'**API du fournisseur cloud** (AWS, Azure, GCP…), par exemple pour créer un load balancer ou gérer les machines. Absent sur un cluster local.

Du déploiement au Pod : le parcours d'une demande

1. Le développeur construit l'image de son application et écrit une **spécification de déploiement** (fichier YAML décrivant l'état voulu).
2. Il l'envoie au cluster (avec `kubectl`) : le `kube-api-server` la reçoit et l'enregistre dans `etcd`.
3. Le `kube-scheduler` repère le nouveau Pod sans Node et choisit le Node le plus adapté.
4. Le `kubelet` de ce Node, informé via le `kube-api-server`, demande au **runtime** de lancer le conteneur : le Pod démarre.
5. `kube-proxy` met en place les règles réseau pour que le Pod soit joignable, et le `kube-controller-manager` veille ensuite à ce que l'état voulu reste respecté.

Cluster

Le **cluster** est l'ensemble formé par le **Control Plane** et les **Nodes de travail** qui exécutent les Pods. C'est à ce cluster que l'on s'adresse avec `kubectl`.

kubectl, l'outil pour piloter Kubernetes

`kubectl` est l'**outil en ligne de commande** qui permet de communiquer avec un cluster Kubernetes : il envoie nos commandes à l'**API Kubernetes** (le `kube-api-server` du Control Plane), et Kubernetes réalise les actions demandées. La documentation officielle le décrit comme un « couteau suisse » : selon la commande, il permet de **créer** des ressources, les **mettre à jour**, les **supprimer**, **afficher** ce qui tourne dans le cluster et **déboguer** quand quelque chose ne va pas.
Exemples : `kubectl get pods` demande à l'API « montre-moi tous les Pods en cours d'exécution » ; `kubectl create deployment …` lui demande « crée ce Deployment pour moi ».
Il n'y a pas de prononciation officielle : « kube C-T-L », « kube control » ou même « kube cuddle » sont tous courants.

Deployment

Un **Deployment** décrit l'**état voulu** d'une application : quelle image lancer et combien de Pods identiques (*réplicas*) doivent tourner. Kubernetes crée ces Pods, les répartit sur les Nodes, et veille en permanence à ce que leur nombre reste celui demandé. Il couvre tout le cycle de vie de l'application :
• **Déployer** (*Deploy*) : créer les Pods à partir de l'image indiquée ;
• **Mettre à l'échelle** (*Scale*) : augmenter ou réduire le nombre de réplicas ;
• **Mettre à jour** (*Update*) : passer à une nouvelle version de l'image en remplaçant les Pods **progressivement**, quelques-uns à la fois, pour que l'application reste disponible pendant toute la mise à jour (*rolling update*) ;
• **Revenir en arrière** (*Rollback*) : si la nouvelle version pose problème, revenir à la version précédente, elle aussi de façon progressive ;
• **Auto-réparation** (*Self-healing*) : si un Pod plante ou disparaît (ex. son Node tombe en panne), le Deployment le **remplace automatiquement** par un nouveau Pod, pour toujours garder le nombre de réplicas demandé ;
• **Suivi de l'état** (*Status check*) : le Deployment indique en permanence où en est l'application : combien de réplicas sont prêts et disponibles, et si une mise à jour est en cours, terminée ou bloquée.

```
kubectl create deployment <nom> \
  --image=<image> \
  --port=<port> \
  --replicas=<nombre>
```

Crée un Deployment : `--image` indique l'image de conteneur à lancer, `--port` le port sur lequel l'application écoute dans le conteneur, et `--replicas` le nombre de Pods identiques à faire tourner en permanence.

Conseil : sans `--replicas`, un seul Pod est créé, et sans `--port` aucun port n'est déclaré. Les options acceptent les deux écritures `--image=<image>` et `--image <image>`. La forme minimale suffit donc : `kubectl create deployment mon-service-`

`deployment --image mon-compte/mon-service:1.0.0`. Le `\` en fin de ligne permet d'écrire la commande sur plusieurs lignes (Linux, macOS, Git Bash).

```
kubectl create deployment mon-service-deployment \
  --image=mon-compte/mon-service:2.0.1 \
  --port=8082 \
  --replicas=3
```

Exemple : déploie l'application à partir de son image publiée sur Docker Hub, qui écoute sur le port `8082`, en 3 exemplaires. Le `kube-scheduler` répartit les 3 Pods sur les Nodes disponibles, et Kubernetes en recrée automatiquement un si l'un d'eux disparaît. Kubernetes confirme la création par `deployment.apps/mon-service-deployment created` (`apps` est le groupe d'API des Deployments, celui de `apiVersion: apps/v1`).

```
kubectl get deployments
```

Liste les Deployments du namespace courant et leur état :
• `READY` : Pods prêts sur Pods demandés (ex. `3/3` : les 3 réplicas sont prêts) ;
• `UP-TO-DATE` : Pods qui utilisent déjà la dernière version du Deployment (utile pendant une mise à jour progressive) ;
• `AVAILABLE` : Pods disponibles pour recevoir du trafic ;
• `AGE` : temps écoulé depuis la création.
Juste après la création, `READY` peut afficher `0/3` le temps que les images soient téléchargées et que les applications démarrent.

Conseil : `kubectl get deploy` est un raccourci équivalent, et `-w` (*watch*) affiche les changements en direct jusqu'à `Ctrl+C`.

```
kubectl get pods
```

Liste les Pods du namespace courant. Les Pods créés par un Deployment portent un nom de la forme `<deployment>-<identifiant>-<suffixe>` (ex. `mon-service-deployment-6989646477-5tbp4`) : l'identifiant du milieu est commun à tous les Pods d'une même version du Deployment, le suffixe est propre à chaque Pod. Les colonnes :
• `READY` : conteneurs prêts sur conteneurs du Pod (ex. `1/1`) ;
• `STATUS` : état du Pod (`Running` = en cours d'exécution ; on peut aussi voir `Pending`, `ContainerCreating`, `CrashLoopBackOff`, `ImagePullBackOff`…) ;
• `RESTARTS` : nombre de redémarrages du conteneur ; une valeur qui augmente signale une application qui plante ;
• `AGE` : temps écoulé depuis la création du Pod.

Conseil : `kubectl get pods -o wide` ajoute l'IP de chaque Pod et le Node sur lequel il tourne. Un statut `ImagePullBackOff` indique en général un nom d'image erroné ou une image introuvable sur le registre. Pour filtrer par label : `kubectl get pods --selector app=<deployment>` (ou `-l`) ne garde que les Pods de ce Deployment, car `create deployment` leur donne le label `app=<nom>`. Ça marche aussi avec `get deployments`. Pour un seul Pod : `kubectl get pod <nom-du-pod>` ; juste après sa création, il affiche `0/1` et `ContainerCreating` le temps de télécharger l'image, puis `1/1` et `Running`.

```
kubectl describe deployment <deployment>
```

Affiche le détail complet d'un Deployment, sous une forme lisible : labels et sélecteur, nombre de réplicas (voulus, à jour, disponibles), stratégie de mise à jour, modèle de Pod (image, ports, variables d'environnement) et, en bas, les événements récents (ex. création ou mise à l'échelle des réplicas). Exemple : `kubectl describe deployment mon-service-deployment`. Les deux écritures fonctionnent : `deployment mon-service-deployment` (avec une espace) ou `deployment/mon-service-deployment` (avec une barre oblique) ; c'est vrai pour la plupart des commandes `kubectl` (`get`, `describe`, `delete`, `set image`…).

Conseil : `describe` fonctionne sur tous les objets (`kubectl describe pod <pod>`, `kubectl describe service <service>`…). C'est le premier réflexe quand quelque chose ne marche pas : la section *Events* indique souvent directement la cause

(image introuvable, ressources insuffisantes...).

```yaml
# mon-service-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-service-deployment
  labels:
    app: mon-service
    tier: backend
…
```

La même chose sous forme de fichier YAML (*manifeste*), la façon habituelle de décrire un Deployment : le fichier se versionne avec le code et se réapplique à l'identique. Le début de tout manifeste :
• `apiVersion` : la version de l'API Kubernetes qui définit ce type d'objet (`apps/v1` pour un Deployment) ;
• `kind` : le type d'objet décrit (`Deployment`) ;
• `metadata` : l'identité de l'objet, avec son `name` (unique) et ses `labels`, des paires clé-valeur libres (ex. `app: mon-service`, `tier: backend`) qui servent à classer et à sélectionner les objets.

Conseil : les labels ne sont pas décoratifs : c'est grâce à eux que Kubernetes relie un Deployment à ses Pods, ou un Service aux Pods vers lesquels envoyer le trafic. Choisis-les cohérents d'un fichier à l'autre.

Service

Le problème que résout un Service

Une application cliente située **hors du cluster** peut joindre un Node par son adresse (ex. `192.168.0.1`), mais **pas directement un Pod** par son IP (ex. `10.1.0.160`) : les IP des Pods appartiennent au réseau interne du cluster et ne sont pas accessibles de l'extérieur. Ce réseau des Pods s'étend sur **tous les Nodes** du cluster : un Pod du Node 1 (`10.1.0.160`) peut joindre directement un Pod du Node 2 (`10.1.0.162`), mais ce réseau reste fermé au monde extérieur.
Et même à l'intérieur du cluster, s'appuyer sur l'IP d'un Pod est fragile : les Pods sont éphémères, un Pod recréé change d'IP, et un Deployment en fait tourner plusieurs (`10.1.0.160`, `10.1.0.157`, `10.1.0.162`...) répartis sur différents Nodes. Il faut donc un **point d'accès stable** qui sache vers quels Pods envoyer le trafic : c'est le rôle du **Service**.

Répartition de charge

Le client n'appelle plus un Pod précis, mais le **Service**, qui a une adresse et un nom **stables**. Le Service connaît en permanence la liste des Pods qui correspondent à son application (retrouvés grâce à leurs *labels*) et **répartit les requêtes** entre eux (ex. tantôt vers `10.1.0.160`, tantôt vers `10.1.0.157`). Le Service joue ainsi le rôle de **répartiteur de charge** (*load balancer*), avec une répartition **aléatoire** par défaut : chaque nouvelle connexion part vers l'un des Pods disponibles, choisi au hasard, ce qui équilibre la charge en moyenne. Si un Pod disparaît ou si de nouveaux réplicas sont ajoutés, le Service met sa liste à jour automatiquement : le client, lui, ne change rien.

Comment un Service trouve ses Pods : les sélecteurs

Un Service ne connaît pas ses Pods par leur nom ni par leur IP : il les retrouve grâce à un **sélecteur** (*selector*), c'est-à-dire un ou plusieurs **labels** que les Pods doivent porter. Exemple : le Service *Frontend* a le sélecteur `tier: frontend` et envoie le trafic à tous les Pods étiquetés `tier: frontend` ; le Service *Backend*, avec `tier: backend`, ne vise que les Pods `tier: backend`. Tout Pod qui reçoit le bon label (par exemple un nouveau réplica) est automatiquement pris en compte par le Service.
Concrètement, le Service tient à jour une **liste d'endpoints** : les adresses IP des Pods qui correspondent à son sélecteur (ex. `10.1.0.161` et `10.1.0.162` pour le Service *Backend*). C'est vers ces adresses qu'il envoie le trafic. Quand un Pod est créé, supprimé ou recréé avec une nouvelle IP, la liste est mise à jour automatiquement.
Enfin, le Service n'envoie le trafic qu'aux **Pods en bonne santé**, prêts à répondre. C'est la **sonde de disponibilité** (*readiness probe*) qui le détermine : un test que Kubernetes exécute régulièrement sur chaque Pod (ex. un appel HTTP à une URL de santé de l'application). Tant que le test échoue, par exemple pendant le

démarrage d'une application Spring Boot, le Pod est retiré de la liste d'endpoints et ne reçoit aucune requête ; il y est réintégré dès que le test réussit.

Communication interne et externe

Le Service est indispensable dans les deux sens :
• **Communication interne** : entre Pods du même cluster. Les Pods *frontend* n'appellent pas directement les IP des Pods *backend* (`10.1.0.161`, `10.1.0.162`), qui peuvent changer : ils appellent le **Service** du backend, qui transmet la requête à l'un des Pods backend disponibles.
• **Communication externe** : depuis l'extérieur du cluster. Une application cliente (navigateur, application mobile, autre système) ne peut pas joindre les IP des Pods : elle passe par un **Service** exposé à l'extérieur, qui reçoit la requête et la dirige vers les Pods concernés (frontend ou backend).
Dans les deux cas, le Service offre le même avantage : un point d'accès stable et une répartition de charge, quels que soient le nombre de Pods et leurs adresses.

Les types de Services

• **`ClusterIP`** : accès **interne au cluster uniquement**. Le Service reçoit une IP virtuelle et un nom DNS joignables seulement depuis les Pods du cluster. C'est le type par défaut, utilisé pour la communication entre microservices (ex. frontend → backend).
• **`NodePort`** : accès **externe via un port des Nodes**. Kubernetes ouvre le même port (par défaut entre `30000` et `32767`) sur chaque Node du cluster ; une application cliente joint le Service à l'adresse `<IP d'un Node>:<NodePort>`, et la requête est transmise aux Pods. Le port est le même sur tous les Nodes (ex. `192.168.0.1:30080` et `192.168.0.2:30080`), et la requête peut atteindre un Pod situé sur **n'importe quel Node**, pas seulement celui qui l'a reçue.
• **`LoadBalancer`** : accès **externe via un répartiteur de charge du fournisseur cloud**. Sur un cluster cloud (AWS, Azure, GCP...), Kubernetes demande automatiquement au fournisseur, via le `cloud-controller-manager`, de créer un load balancer avec une **adresse IP publique** ; les clients utilisent cette adresse unique, et le load balancer répartit le trafic entre les Nodes du cluster. C'est le moyen le plus courant d'exposer une application en production.
• **`ExternalName`** : **alias DNS vers un service externe** au cluster. Le Service ne pointe vers aucun Pod : son nom renvoie simplement vers un nom de domaine extérieur (ex. une base de données hébergée dans le cloud). Exemple : un Service `ExternalName` nommé `ma-base-db` renvoie vers `ma-base.rds.amazonaws.com` (base AWS RDS) ; l'application appelle simplement `ma-base-db`, comme n'importe quel service interne, et le DNS du cluster la redirige vers l'adresse externe ; si l'adresse externe change, seul le Service est à modifier, pas le code ni la configuration des applications.

Créer un Service en ligne de commande

```
kubectl expose deployment <deployment> --type=<type>
```

Crée un Service devant les Pods d'un Deployment, en mode impératif. Le Service reprend automatiquement le sélecteur du Deployment, et donc cible ses Pods. `--type` choisit le type de Service :
• `ClusterIP` (par défaut) : accès interne au cluster uniquement ;
• `NodePort` : accès externe via `<IP du Node>:<NodePort>`, le port étant attribué automatiquement entre `30000` et `32767` ;
• `LoadBalancer` : accès externe via un load balancer (avec Docker Desktop, il est joignable sur `localhost`).

Conseil : sans `--port`, le Service reprend le port déclaré dans le Deployment (`--port=8082` à sa création) ; si aucun port n'a été déclaré, `expose` échoue et il faut le préciser.

```
kubectl expose deployment mon-service-deployment \
  --type=LoadBalancer \
  --port=8082 \
  --target-port=8082 \
  --name mon-service-service
```

Exemple complet :
• `--port` : le port du Service, celui que les clients appellent (on peut en choisir un différent si besoin) ;
• `--target-port` : le port du conteneur vers lequel le trafic est transmis, celui sur lequel l'application écoute

(`containerPort`, ou `server.port` pour Spring Boot) ;
• `--name` : le nom du Service (sinon il prend celui du Deployment).
Kubernetes confirme par `service/mon-service-service exposed`.

Conseil : comme pour Docker (`-p hôte:conteneur`), `--port` et `--target-port` peuvent différer, ex. `--port=80 --target-port=8082` pour appeler l'application sur le port 80.

```
kubectl get services
```

Liste les Services du namespace courant. Les colonnes :
• `TYPE` : `ClusterIP`, `NodePort`, `LoadBalancer`... ;
• `CLUSTER-IP` : l'IP interne stable du Service, joignable depuis les Pods (ex. `10.96.241.158`) ;
• `EXTERNAL-IP` : l'adresse externe attribuée par le load balancer (ex. `172.18.0.5`), `<none>` pour un `ClusterIP`, ou `<pending>` tant qu'elle n'est pas encore attribuée ;
• `PORT(S)` : le port du Service et, pour `NodePort` et `LoadBalancer`, le NodePort ouvert sur les Nodes (ex. `8082:31270/TCP` : port `8082` du Service, port `31270` sur chaque Node) ;
• `AGE` : temps écoulé depuis la création.
Le Service `kubernetes` (`10.96.0.1`, `443/TCP`) existe d'office : c'est l'accès à l'API du cluster.

Conseil : un Service `LoadBalancer` crée aussi un NodePort, car le load balancer s'appuie sur lui pour atteindre les Nodes. Avec Docker Desktop, l'`EXTERNAL-IP` (ex. `172.18.0.5`) est une adresse interne à la machine virtuelle de Docker Desktop, pas toujours joignable directement depuis ta machine ; l'application est en général joignable sur `http://localhost:8082` : c'est là qu'il faut relancer la requête de test (`curl` ou Postman) préparée plus haut. Une réponse `200 OK` avec un `userId` généré confirme que toute la chaîne fonctionne : requête → Service `LoadBalancer` → l'un des 3 Pods du Deployment. `kubectl get svc` est un raccourci équivalent.

Installer un cluster local

Pour s'entraîner, **Docker Desktop** peut créer un cluster Kubernetes sur sa propre machine : menu **Kubernetes** → **Create Kubernetes Cluster**. Deux types de cluster sont proposés :
• **Kubeadm** : crée un cluster à **un seul Node** avec l'outil officiel `kubeadm`. Le plus simple pour débuter.
• **kind** (*Kubernetes in Docker*) : crée un cluster contenant **un ou plusieurs Nodes**, chacun étant un conteneur Docker. Le curseur *Node(s)* permet d'en choisir le nombre (jusqu'à 10), pratique pour observer la répartition des Pods sur plusieurs Nodes. Nécessite d'activer le *containerd image store* dans les paramètres de Docker Desktop.
On peut aussi choisir la **version de Kubernetes**. Changer le nombre de Nodes ou la version **réinitialise le cluster** : tout ce qui y était déployé est supprimé. L'option *Show system containers* affiche les conteneurs internes de Kubernetes dans les commandes Docker (utile seulement pour l'exploration). Docker Desktop installe aussi `kubectl` et le configure automatiquement pour ce cluster.

```
kubectl get nodes
```

Vérification une fois le cluster créé : liste les Nodes du cluster avec leur statut. Un Node prêt affiche `Ready` ; avec un cluster à un seul Node, une seule ligne apparaît.

Conseil : `kubectl config current-context` indique sur quel cluster pointe `kubectl` (ex. `docker-desktop`) : utile si tu travailles avec plusieurs clusters.

Une fois le cluster lancé, l'écran **Kubernetes** de Docker Desktop sert de tableau de bord : état du cluster (`Active`), type (`kind` ou `kubeadm`), nombre de Nodes et version, puis la liste des **Deployments**, **Pods**, **Nodes**, **Services**, **Ingresses** (règles d'accès HTTP depuis l'extérieur) et **ConfigMaps / Secrets** (configuration et données sensibles des applications) du namespace choisi (`default` par défaut). Les boutons *Stop* et *Edit cluster* arrêtent ou modifient le cluster.
Sur un cluster neuf, on y voit déjà deux éléments créés automatiquement : le Node `desktop-control-plane` (statut `Ready`), qui joue à la fois le rôle de Control Plane et de Node de travail dans un cluster à un seul Node ; et le Service `kubernetes` (Cluster IP `10.96.0.1`, port `443/TCP`), qui donne accès à l'API Kubernetes (`kube-api-server`) depuis l'intérieur du cluster. La ConfigMap `kube-root-ca.crt`, présente elle aussi d'office, contient le certificat racine du cluster, qui permet aux Pods de vérifier qu'ils parlent bien au vrai `kube-api-server`. Il ne faut pas supprimer ces éléments.

```
kubectl run <nom-du-pod> --image <image>
```

Crée et lance un seul Pod, directement, sans Deployment. Exemple proposé par Docker Desktop : `kubectl run nginx --image nginx` lance un Pod `nginx` à partir de l'image officielle du serveur web nginx.

Conseil : pratique pour un test rapide, mais un Pod lancé ainsi n'est pas recréé s'il disparaît, et ne peut ni être mis à l'échelle ni mis à jour progressivement. Pour une vraie application, préfère `kubectl create deployment` (ex. `kubectl create deployment my-app --image=nginx`).

Préparer l'image de l'application

Kubernetes ne construit pas d'image : il lance des conteneurs à partir d'images existantes. Avant de déployer une application Spring Boot, il faut donc d'abord **construire son image Docker**, puis la rendre accessible au cluster.

```
# Stage 1 : dépendances
FROM ${BASE_JDK_IMAGE} AS dependencies
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline

# Stage 2 : build
FROM dependencies AS builder
COPY src ./src
RUN mvn clean package

# Stage 3 : runtime
FROM ${BASE_JRE_IMAGE} AS runtime
# … copie du .jar depuis builder, puis lancement de l'application
```

Un Dockerfile multi-stage découpe la construction en étapes (*stages*), chacune partant d'une image (`FROM … AS <nom>`) et pouvant réutiliser la précédente :
• `dependencies` : copie seulement le `pom.xml` et télécharge les dépendances Maven (`mvn dependency:go-offline`) ;
• `builder` : repart de `dependencies`, copie le code source et compile l'application (`mvn clean package`), ce qui produit le `.jar` ;
• `runtime` : part d'une image JRE légère (sans Maven ni JDK), y copie seulement le `.jar` et lance l'application.
Seule la dernière étape forme l'image finale, beaucoup plus petite qu'une image contenant tous les outils de build.

Conseil : copier le `pom.xml` avant le code source permet à Docker de garder les dépendances en cache : tant que le `pom.xml` ne change pas, une modification du code ne relance pas leur téléchargement. `${BASE_JDK_IMAGE}` et `${BASE_JRE_IMAGE}` sont des arguments de build (`ARG`) qui permettent de changer l'image de base sans modifier le Dockerfile.

```
docker build -f Dockerfile-multi-stage -t mon-service:1.0.0 .
```

Construit l'image à partir du Dockerfile multi-stage (`-f`), la nomme `mon-service:1.0.0` (`-t`), avec le dossier courant (`.`) comme contexte de build. À lancer depuis la racine du projet, là où se trouvent le `pom.xml` et le dossier `src`.

Conseil : avec le cluster local de Docker Desktop, une image construite localement est en général utilisable directement. Pour un cluster distant, il faut d'abord la publier sur un registre (ex. `docker push` vers Docker Hub).

```
docker images
```

Vérification : l'image construite apparaît dans la liste (ex. `mon-service:1.0.0`), avec son ID et sa taille. Avec un cluster kind de Docker Desktop, on y voit aussi les images du cluster lui-même : `kindest/node` (l'image des

Nodes kind), `desktop-containerd-registry-mirror` et `desktop-cloud-provider-kind`. La colonne *EXTRA* les marque `U` (*In Use*) car elles sont utilisées par des conteneurs en cours d'exécution.

Conseil : ne supprime pas ces images de Docker Desktop tant que le cluster est actif (attention à `docker image prune -a` ou `docker system prune -a`).

```
docker run -p 8082:8082 --name mon-service mon-service:1.0.0
```

Test de l'image avant de la déployer dans Kubernetes : la lance dans un simple conteneur Docker, avec le port `8082` publié sur la machine. Si la bannière Spring Boot s'affiche et que l'application démarre sans erreur, l'image fonctionne ; tout problème découvert ici serait aussi survenu dans le cluster, mais y aurait été plus difficile à diagnostiquer.

Conseil : une fois les tests terminés, arrête et supprime ce conteneur (`docker rm -f mon-service`) avant de passer à Kubernetes, pour libérer le port `8082`.

```
curl -X POST http://localhost:8082/users \
  -H 'Content-Type: application/json' \
  -d
'{"firstName":"Jean","lastName":"Dupont","email":"jean.dupont@example.com","password":"motd
epasse123"}'
```

Vérification fonctionnelle du conteneur de test : envoie une requête `POST` à l'application sur le port publié pour créer un utilisateur. Une réponse `200 OK` avec un JSON contenant un `userId` généré confirme que l'application répond correctement. On peut faire la même requête avec Postman (méthode `POST`, onglet *Body* → *raw* → *JSON*).

Conseil : garde cette requête sous la main, elle servira à vérifier de la même façon l'application une fois déployée dans Kubernetes.

```
docker images                                          # 1. repère l'ID de l'image
docker tag d0ff1769fcd3 mon-compte/mon-service:1.0.0   # 2. la nomme au format Docker
Hub
docker login --username=mon-compte                     # 3. se connecte à Docker Hub
docker push mon-compte/mon-service:1.0.0               # 4. publie l'image
```

Publication de l'image sur Docker Hub, pour qu'un cluster puisse la télécharger : `docker tag` donne à l'image locale (désignée par son ID) un nom au format `<compte>/<nom>:<tag>`, `docker login` ouvre la session (le mot de passe est demandé ensuite, `Login Succeeded` confirme), puis `docker push` envoie l'image. C'est ce nom complet (ex. `mon-compte/mon-service:1.0.0`) qu'on indique ensuite dans `--image` du Deployment.

Conseil : comme le suggère Docker, utilise un jeton d'accès personnel (*Personal Access Token*, créé dans les paramètres du compte Docker Hub) à la place du mot de passe. La procédure est détaillée dans l'aide-mémoire Docker (section « Build et gestion des images »). Un push réussi se termine par une ligne `<tag>: digest: sha256:… size: …`. Les couches déjà présentes sur Docker Hub ne sont pas renvoyées : `Mounted from <autre dépôt>` indique une couche réutilisée depuis un autre dépôt du même compte, `Pushed` une couche réellement envoyée. Pour une nouvelle version, on reconstruit, on tague avec un nouveau numéro (ex. `1.0.1`) et on pousse : c'est ce tag qui servira à mettre à jour le Deployment.

Les objets de l'API Kubernetes

Tout ce qu'on crée dans un cluster est un **objet de l'API Kubernetes**, décrit par un `kind` dans un manifeste YAML et géré via le `kube-api-server`. Les principaux :

- **`Pod`** : fait tourner le conteneur de l'application ;
- **`Deployment`** : gère les réplicas d'un Pod (nombre, mises à jour, retours arrière, auto-réparation) ;
- **`Service`** : donne un accès réseau stable aux Pods ;
- **`ConfigMap`** : stocke des données de configuration **non confidentielles** sous forme de paires clé-valeur (ex.

profil Spring actif, URL d'un autre service), injectées dans les Pods comme variables d'environnement ou fichiers ;
• **`Secret`** : stocke des informations **sensibles** comme des mots de passe ou des clés d'API, pour ne pas les écrire en clair dans l'image ni dans le manifeste du Deployment.
`ConfigMap` et `Secret` jouent le même rôle que le fichier `.env` et les *secrets* de Docker Compose : séparer la configuration du code, pour utiliser la même image dans tous les environnements.

Approche impérative ou déclarative

Approche impérative

On donne à Kubernetes des **ordres directs**, une commande `kubectl` par action : créer un Deployment, changer son image, modifier son nombre de réplicas...

```
kubectl set image deployment/<deployment> <conteneur>=<nouvelle_image>
```

Change l'image d'un conteneur d'un Deployment, ce qui déclenche une mise à jour progressive (*rolling update*) : les Pods sont remplacés quelques-uns à la fois par des Pods utilisant la nouvelle image. Kubernetes confirme par `deployment.apps/<deployment> image updated` ; `kubectl rollout status deployment/<deployment>` suit ensuite la progression jusqu'à la fin du remplacement. Exemple : `kubectl set image deployment/mon-service mon-service=mon-service:2.0.2`.

Conseil : le nom avant le `=` est celui du conteneur dans le Pod, pas celui du Deployment. Avec `kubectl create deployment`, le conteneur prend le nom de l'image, sans le compte ni le tag : pour `--image mon-compte/mon-service-demo:1.0.0`, il s'appelle `mon-service-demo`. Exemple : `kubectl set image deployment/mon-service-deployment mon-service-demo=mon-compte/mon-service-demo:1.0.1`. Pour retrouver le nom du conteneur : `kubectl describe deployment <deployment>` (rubrique *Containers*). Le type d'objet est obligatoire avant le nom, sous la forme `deployment/<nom>` ou `deployment <nom>` : sans lui, `kubectl` prend le nom du Deployment pour un type de ressource et répond par une erreur (`the server doesn't have a resource type …`).

```
kubectl rollout history deployment/<deployment>
```

Affiche l'historique des révisions d'un Deployment : chaque changement du modèle de Pod (ex. nouvelle image avec `set image`) crée une nouvelle révision numérotée (`1` = création, `2` = première mise à jour...). Ce sont ces révisions qui permettent de revenir en arrière.

Conseil : la colonne `CHANGE-CAUSE` affiche `<none>` par défaut ; pour la remplir, voir `kubectl annotate` ci-dessous. `kubectl rollout history deployment/<deployment> --revision=2` affiche le détail d'une révision (image, ports...).

```
kubectl annotate deployment/mon-service-deployment \
  kubernetes.io/change-cause="Mise à jour de l'image vers mon-compte/mon-service:1.0.1"
```

Ajoute une annotation au Deployment : une paire clé-valeur d'information, qui (contrairement aux labels) ne sert pas à sélectionner des objets. L'annotation `kubernetes.io/change-cause` est particulière : sa valeur s'affiche dans la colonne `CHANGE-CAUSE` de `kubectl rollout history`, en face de la révision la plus récente. Ici, `kubectl rollout history` affiche ensuite `2  Mise à jour de l'image vers mon-compte/mon-service:1.0.1`, la révision `1` restant à `<none>`. Kubernetes confirme par `deployment.apps/<deployment> annotated`.

Conseil : l'annotation décrit la dernière révision ; lance-la donc juste après chaque `set image`, avant le changement suivant. Pour modifier une annotation existante, ajoute `--overwrite`. Attention : l'annotation reste sur le Deployment, donc une nouvelle révision (par exemple après un `set env`) reprend l'ancienne CHANGE-CAUSE : la révision `3` afficherait encore `Mise à jour de l'image vers mon-compte/mon-service:1.0.1`. Après chaque changement, mets donc à jour l'annotation, par exemple après le `set env` : `kubectl annotate deployment/mon-service-deployment kubernetes.io/change-cause="Variables d'environnement : SPRING_PROFILES_ACTIVE=test, LOG_LEVEL=DEBUG" --overwrite`. La révision `3` affiche alors ce texte.

```
kubectl set env deployment/<deployment> <VAR>=<valeur> <VAR2>=<valeur2>
```

Ajoute ou modifie des variables d'environnement dans les conteneurs d'un Deployment, sans reconstruire l'image. Exemple : `kubectl set env deployment/mon-service-deployment SPRING_PROFILES_ACTIVE=test LOG_LEVEL=DEBUG` active le profil Spring `test` et passe les logs en mode détaillé. Kubernetes confirme par `deployment.apps/<deployment> env updated`.

Conseil : changer une variable modifie le modèle de Pod, donc déclenche une mise à jour progressive (nouveaux Pods) et crée une nouvelle révision dans `rollout history`. `kubectl set env deployment/<deployment> --list` affiche les variables actuelles, et `<VAR>-` (avec un tiret final) supprime une variable : `kubectl set env deployment/mon-service-deployment LOG_LEVEL-` retire `LOG_LEVEL` (réponse `env updated`, et nouvelle révision elle aussi). Pour vérifier, `kubectl describe deployment <deployment>` liste les variables dans la rubrique `Environment` du `Pod Template`. En bas de la sortie, `NewReplicaSet` indique le ReplicaSet actif (ex. 3/3 replicas) et `OldReplicaSets` les anciens, gardés à 0 replica : c'est ce qui permet un retour arrière avec `rollout undo`.

`kubectl set env pods --all --list`

Affiche, sans rien modifier, les variables d'environnement de chaque Pod. La sortie commence pour chaque Pod par une ligne `# Pod <nom-du-pod>, container <conteneur>`, suivie des variables au format `VAR=valeur`. `--all` cible tous les Pods du namespace, `--list` se contente de lister.

Conseil : pratique après un `set env` pour vérifier que les nouveaux Pods (nouveau suffixe de ReplicaSet dans leur nom) ont bien reçu les valeurs. Pour un seul Pod : `kubectl set env pod/<nom-du-pod> --list`.

`kubectl rollout undo deployment/<deployment>`

Retour arrière (rollback) : revient à la révision précédente du Deployment. Kubernetes réactive l'ancien ReplicaSet (celui listé dans `OldReplicaSets`) avec une mise à jour progressive, sans interruption de service. Exemple : `kubectl rollout undo deployment/mon-service-deployment`. Kubernetes confirme par `deployment.apps/<deployment> rolled back`.

Conseil : la révision restaurée n'est pas recopiée, elle est renumérotée. Après un retour de la révision 4 vers la 3, `rollout history` n'affiche plus la 3 mais une révision `5` avec la CHANGE-CAUSE de la 3. Pour viser une révision précise : `kubectl rollout undo deployment/<deployment> --to-revision=2`. Pour suivre le retour arrière : `kubectl rollout status deployment/<deployment>`, qui affiche `deployment "<deployment>" successfully rolled out` une fois tous les Pods remplacés.

`kubectl scale deployment <deployment> --replicas=<nombre>`

Modifie le nombre de réplicas d'un Deployment : Kubernetes crée ou supprime des Pods pour atteindre ce nombre. Exemple : `kubectl scale deployment mon-service --replicas=5`. Kubernetes confirme par `deployment.apps/<deployment> scaled`, et `kubectl get deployments` affiche ensuite le nouveau nombre dans la colonne `READY` (ex. `1/1` après `--replicas=1`).

Conseil : l'objet visé s'écrit au choix `<type> <nom>` ou `<type>/<nom>` : `kubectl scale deployment/mon-service --replicas=5` est équivalent. Ces deux écritures valent pour les autres commandes `kubectl` qui visent un objet (`describe`, `set image`, `set env`, `rollout`, `annotate`…). Seul le nom, sans le type, ne suffit pas. Contrairement à `set image` ou `set env`, `scale` ne modifie pas le modèle de Pod : il ne crée pas de nouvelle révision dans `rollout history`.

`kubectl logs -f deployment/<deployment> --all-pods`

Affiche les logs des conteneurs. `-f` (`--follow`) suit les logs en direct, comme `docker logs -f` (`Ctrl+C` pour arrêter). Avec `deployment/<deployment>`, `--all-pods` réunit les logs de tous les Pods du Deployment. Exemple : `kubectl logs -f deployment/mon-service-deployment --all-pods`.

Conseil : sans `--all-pods`, `kubectl logs deployment/<deployment>` ne montre qu'un seul Pod, choisi par Kubernetes. Ajoute `--prefix` pour savoir de quel Pod vient chaque ligne. Pour un Pod précis : `kubectl logs -f <nom-du-pod>` (nom donné par `kubectl get pods`). Si ce Pod contient plusieurs conteneurs, `--all-containers=true` affiche les logs de tous, et `-c <conteneur>` en choisit un seul. `--all-pods` demande une version récente de `kubectl` ; sur une ancienne version, utilise un sélecteur de labels : `kubectl logs -f -l app=<deployment>`.

Avantages : plus rapide pour déployer de petits projets, sans fichiers YAML à écrire ; plus rapide pour faire des modifications ; idéal pour apprendre et expérimenter.
Inconvénients : il faut retenir des commandes différentes pour chaque type d'objet ; pas de gestion de versions, donc difficile de suivre les changements ; et surtout, **difficile de reproduire la même configuration** (sur un autre cluster ou après une réinstallation), puisque l'état final dépend de la suite de commandes tapées.

Approche déclarative

On **décrit l'état voulu** dans des fichiers de configuration (YAML ou JSON), puis on les applique avec `kubectl apply` : c'est Kubernetes qui calcule et effectue les changements nécessaires pour atteindre cet état.

```yaml
# mon-service-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mon-service-pod
  labels:
    app: mon-service
    version: "2.0.1"
spec:
  containers:
    - name: mon-service
      image: mon-service:2.0.1
      ports:
        - containerPort: 8080
      env:
        - name: SPRING_PROFILES_ACTIVE
          value: "default"
```

Exemple de manifeste décrivant un Pod. Après `apiVersion` (`v1` pour un Pod), `kind` et `metadata`, la section `spec` décrit le contenu voulu : la liste `containers` (ici un seul conteneur), avec pour chacun son `name`, son `image`, ses `ports` (`containerPort` : le port sur lequel l'application écoute dans le conteneur) et ses variables d'environnement `env`, sous forme de paires `name` / `value`.

Conseil : les valeurs qui ressemblent à des nombres mais doivent rester du texte (versions, profils…) s'écrivent entre guillemets, ex. `version: "2.0.1"`. En pratique, on décrit rarement un Pod seul : on écrit un Deployment, dont la partie `template` contient exactement ce même bloc de Pod (`containers`, `ports`…). Un Pod seul reste pratique pour tester une image avant d'écrire le Deployment : vérifier qu'elle se télécharge, qu'elle démarre et que sa configuration fonctionne. Sous le conteneur, `imagePullPolicy` indique quand télécharger l'image : `Always` la vérifie sur le registre à chaque démarrage de conteneur, `IfNotPresent` réutilise l'image locale si elle existe déjà, et `Never` n'utilise que l'image locale (le Pod échoue avec `ErrImageNeverPull` si elle est absente). Sans cette ligne, la valeur par défaut est `Always` pour le tag `latest` (ou sans tag), et `IfNotPresent` pour un tag précis comme `2.0.1`.

```
kubectl apply -f <fichier.yaml>
```

Applique un manifeste : Kubernetes compare l'état décrit dans le fichier à l'état actuel du cluster, puis crée les objets qui n'existent pas encore ou met à jour ceux qui ont changé. La même commande sert donc à la création comme aux modifications : on modifie le fichier, puis on relance `apply`.

Conseil : `-f` accepte aussi un dossier entier (`kubectl apply -f k8s/`), pratique pour appliquer tous les manifestes d'une application en une fois. À l'inverse, `kubectl delete -f <fichier.yaml>` (ou un dossier) supprime les objets décrits. `kubectl create -f <fichier.yaml>` crée aussi les objets (`pod/<nom> created`), mais seulement s'ils n'existent pas : relancée, elle échoue avec `AlreadyExists`, alors qu'`apply` met à jour. Pour l'approche déclarative, préfère `apply`, de la création aux modifications.

```
kubectl apply --dry-run=server -f <fichier.yaml>
```

Test à blanc : envoie le manifeste au serveur d'API, qui le valide entièrement comme pour un vrai `apply`, mais n'enregistre rien dans le cluster. Exemple : `kubectl apply --dry-run=server -f mon-service-pod.yaml` répond `pod/mon-service-pod created (server dry run)` : le fichier est valide, mais aucun Pod n'a été créé.

Conseil : à lancer avant un vrai `apply` pour repérer une faute de frappe ou un champ invalide. `--dry-run=client` ne contacte pas le cluster et vérifie donc moins de choses ; il sert surtout à générer un manifeste à partir d'une commande impérative : `kubectl create deployment mon-service-deployment --image mon-compte/mon-service:1.0.0 --dry-run=client -o yaml > mon-service-deployment.yaml`.

Exemple complet : une application Spring décrite en quatre manifestes (ConfigMap, Secret, Deployment, Service), rangés dans un dossier `kubernetes/` du projet et appliqués d'un coup avec `kubectl apply -f kubernetes/`.

```yaml
# mon-service-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: mon-service-config
data:
  server.port: "8082"
  spring.profiles.active: default
  spring.h2.console.enabled: "true"
  spring.datasource.url: jdbc:h2:mem:testdb
  spring.jpa.hibernate.ddl-auto: update
  logging.file.name: logs/mon-service.log
```

Manifeste de ConfigMap : sous `data`, des paires clé-valeur de configuration non confidentielle. Ici, les clés sont directement des propriétés Spring (celles d'`application.properties`) : injectées dans le conteneur par le Deployment, elles remplacent les valeurs de l'image sans la reconstruire.

Conseil : toutes les valeurs d'une ConfigMap sont du texte : les nombres et booléens s'écrivent entre guillemets (`"8082"`, `"true"`), sinon `apply` refuse le fichier. Pour changer la configuration, on modifie le fichier puis on relance `kubectl apply`, puis `kubectl rollout restart deployment/<deployment>` : des variables d'environnement déjà injectées ne sont pas relues par les Pods en cours.

```yaml
# mon-service-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: mon-service-secret
type: Opaque
data:
  spring.datasource.username: dXRpbGlzYXRldXI=
  spring.datasource.password: bW90ZGVwYXNzZQ==
```

Manifeste de Secret. `type: Opaque` désigne un Secret générique, fait de paires clé-valeur libres. Sous `data`, chaque valeur est encodée en base64 (ici `utilisateur` et `motdepasse`). Pour encoder une valeur : `echo -n 'motdepasse' | base64` ; pour la relire : `echo 'bW90ZGVwYXNzZQ==' | base64 -d`.

Conseil : base64 est un encodage, pas un chiffrement : quiconque lit le fichier retrouve la valeur. Ne versionne donc pas un Secret contenant de vrais mots de passe. Le `-n` d'`echo` évite d'encoder le retour à la ligne final, qui fausserait le mot de passe. Avec `stringData` à la place de `data`, on écrit les valeurs en clair et Kubernetes les encode lui-même.

```yaml
# mon-service-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-service-deployment
```

```yaml
  labels:
    app: mon-service
    tier: backend
  annotations:
    kubernetes.io/change-cause: "Image 1.0.2, secrets montés en volume"
spec:
  replicas: 4
  selector:
    matchLabels:
      app: mon-service
  template:
    metadata:
      labels:
        app: mon-service
    spec:
      containers:
        - name: mon-service-container
          image: mon-compte/mon-service:1.0.2
          ports:
            - containerPort: 8082
          envFrom:
            - configMapRef:
                name: mon-service-config
          volumeMounts:
            - name: db-secrets-volume
              mountPath: /opt/secrets/db
      volumes:
        - name: db-secrets-volume
          secret:
            secretName: mon-service-secret
```

Manifeste de Deployment (`apiVersion: apps/v1`) :
• `metadata.annotations` : la `change-cause` s'écrit dans le fichier ; elle s'affiche dans `rollout history` après l'`apply` ;
• `spec.replicas` : nombre de Pods à faire tourner ;
• `spec.selector.matchLabels` : les Pods que gère le Deployment ; il doit correspondre aux `labels` du `template` ;
• `spec.template` : le modèle de Pod, avec le même contenu qu'un manifeste de Pod (sans `apiVersion` ni `kind`) ;
• `envFrom` + `configMapRef` : injecte toutes les clés de la ConfigMap comme variables d'environnement ;
• `volumes` + `volumeMounts` : le Secret devient un volume monté dans `/opt/secrets/db` ; chaque clé devient un fichier (ex. `/opt/secrets/db/spring.datasource.password`) qui contient la valeur décodée.

Conseil : Spring Boot lit ce dossier avec `spring.config.import=optional:configtree:/opt/secrets/db/` : le nom de chaque fichier devient une propriété. Pour injecter plutôt le Secret en variables d'environnement, ajoute `- secretRef:` avec `name: mon-service-secret` sous `envFrom`. La ConfigMap et le Secret doivent exister : tant qu'ils manquent, les Pods ne démarrent pas.

```yaml
# mon-service-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: mon-service-service
  labels:
    app: mon-service
    tier: backend
spec:
  type: LoadBalancer
  selector:
```

```
    app: mon-service
  ports:
    - protocol: TCP
      port: 8082
      targetPort: 8082
```

Manifeste de Service, l'équivalent déclaratif de `kubectl expose` :
- `type` : `ClusterIP` (par défaut), `NodePort` ou `LoadBalancer` ;
- `selector` : le Service envoie le trafic aux Pods qui portent ce label ;
- `ports` : `port` est le port du Service, `targetPort` le port du conteneur (son `containerPort`).

Conseil : le `selector` du Service reprend les labels du `template` du Deployment (ceux des Pods), pas ceux de son `metadata`. Avec le cluster de Docker Desktop, un Service `LoadBalancer` est joignable sur `localhost:<port>`. Les labels comme `tier: backend` servent à classer et filtrer les objets, ex. `kubectl get all -l tier=backend`.

Avantages : adaptée à la **production** ; les fichiers se placent sous **gestion de versions** (Git) avec le code, ce qui permet de suivre et de relire chaque changement ; idéale pour le **travail en équipe**, chacun appliquant les mêmes fichiers.
Inconvénients : plus long à mettre en place ; les fichiers peuvent devenir **obsolètes** ; il faut garder les fichiers et le cluster **synchronisés** (une modification faite en impératif sur le cluster ne se reflète pas dans les fichiers, et sera écrasée au prochain `apply`).